REVERSE YOUR FUTURE

Diving into Firmware Cartography with Pyrrha 2.0

www.quarkslab.com

Eloïse Brocas

Hack.Lu 2026, Luxembourg

22/10/2026



Quarkslab
REVERSE YOUR FUTURE

Our target

tree

```
cryptocorn in ~ λ tree RAX30_rootfs
RAX30_rootfs
├── bin
├── data
├── debug -> sys/kernel/debug
├── dev
├── etc
├── lib
├── libexec
├── mnt
├── opt
├── proc
├── sbin
├── sys
├── tmp -> var/tmp
├── usr
├── var
└── webs

140 directories, 2069 files
```

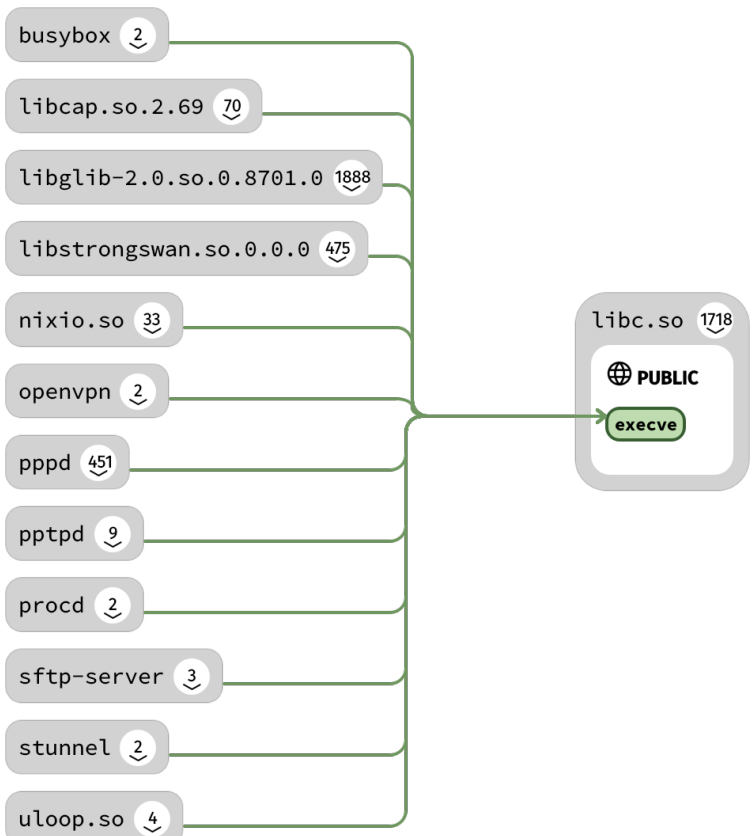


Figure 1: Netgear RAX30

With Pyrrha, you can *now* zoom into the firmware to **follow calls across binaries**

Three Mappers, Three Levels of Detail

fs **Which binary is using this library?**



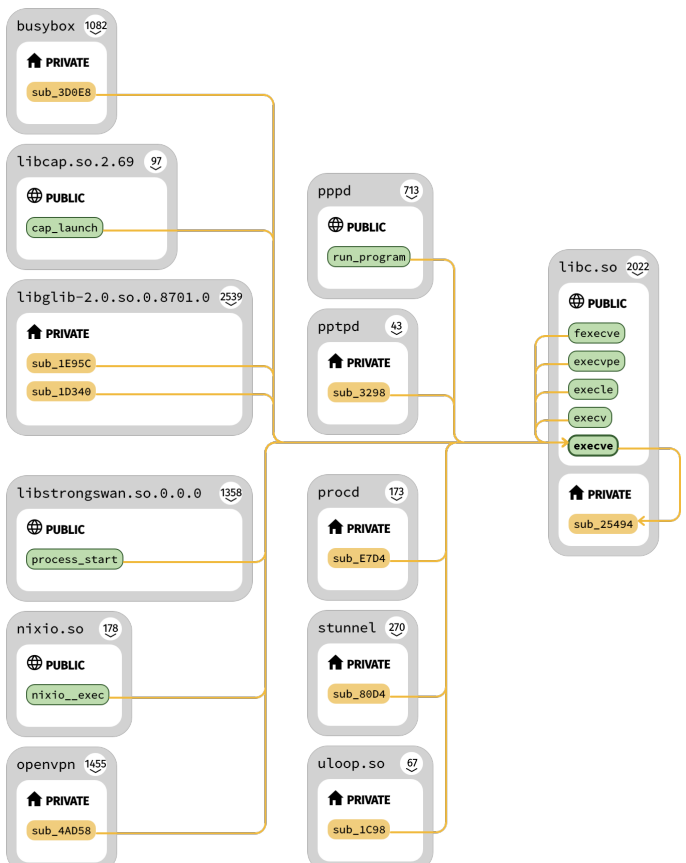
Quarkslab

Three Mappers, Three Levels of Detail

`fs` **Which binary is using this library?**

+ disassembler data

`fs-cg` **Where in the firmware is this function used?**



Quarkslab

Three Mappers, Three Levels of Detail

`fs` **Which binary is using this library?**

+ disassembler data

`fs-cg` **Where in the firmware is this function used?**

focus on one binary

`decomp` **Display code and its interactions**



sub_1D340@0x1d340* 1 reference

```
..   sub_1D340@0x1d340
23     }
24     v9 = *argv;
25     if ( envp )
26       execve(v9, (char *const *)argv, envp);
27     else
28       execv(v9, (char *const *)argv);
29     return (_BYTE *)(&dword_0 + 1);
30   }
```

sub_1E95C@0x1e95c* 2 references

```
...   sub_1E95C@0x1e95c
312               else
313                 v70 = (char *)memcpy((void *)(v65 -
314               if ( envp )
315                 execve(v70, v23, envp);
316               else
317                 execv(v70, v23);
318               if ( *v58 == 8 && !sub_1D340(v70, v23
```


Quarkslab

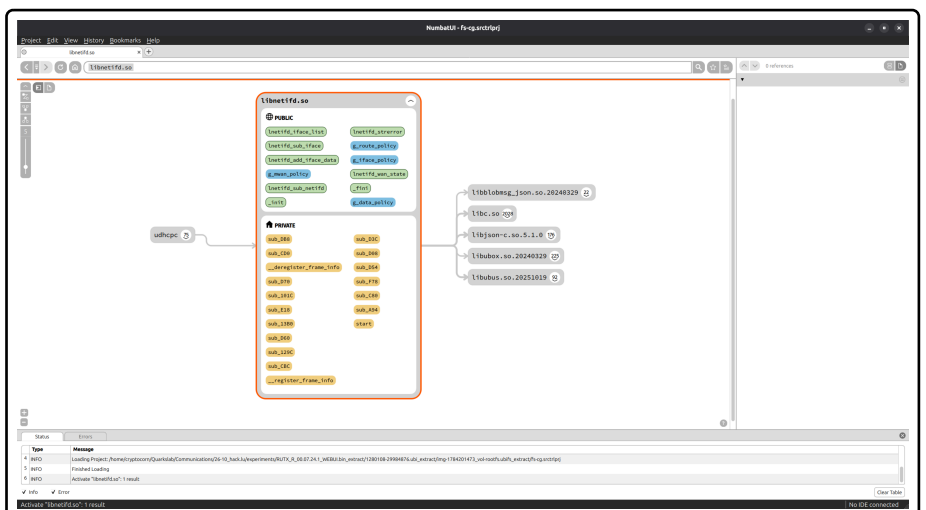
Two step workflow

Pyrrha

```
pyrrha fs-cg

(pyrrha) cryptocorn in ~ λ IDADIR=/opt/ida-pro-9.3 pyrrha fs-cg firmware_rootfs
Binaries mapping ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 747/747 0:03:47
Symlinks mapping ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 470/470 0:00:00
Library imports mapping ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 747/747 0:00:00
Call Graph mapping ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 747/747 0:00:06
```

NumbatUI

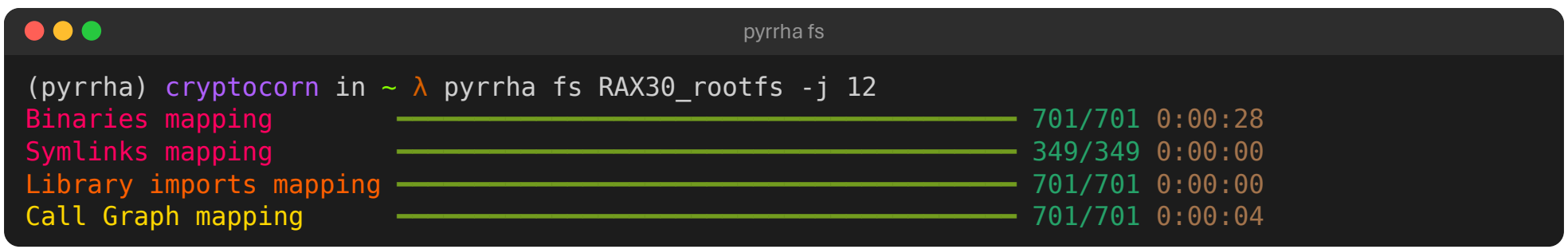


Quarkslab



Demo

Filesystem imports



Quarkslab

Secure vs. insecure TLS with curl

Command line

secure

```
cryptocorn in ~ λ curl URL
```

insecure

```
cryptocorn in ~ λ curl --insecure URL
```

C library

secure

```
curl_easy_setopt(p1, CURLOPT_URL, url);
curl_easy_setopt(p1, CURLOPT_SSL_VERIFYHOST, 1);
curl_easy_setopt(p1, CURLOPT_SSL_VERIFYPEER, 1);
```

insecure

```
curl_easy_setopt(p1, CURLOPT_URL, url);
curl_easy_setopt(p1, CURLOPT_SSL_VERIFYHOST, 0);
curl_easy_setopt(p1, CURLOPT_SSL_VERIFYPEER, 0);
```

Firmware call graph

pyrrha fs-cg

```
(pyrrha) cryptocorn in ~ λ IDADIR=/opt/ida-pro-9.3 pyrrha fs-cg RAX30_rootfs -j 12
Binaries mapping          ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 701/701 0:04:50
Symlinks mapping          ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 349/349 0:00:00
Library imports mapping   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 701/701 0:00:00
Call Graph mapping        ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 701/701 0:00:06
```

pyrrha fs-cg

```
(pyrrha) cryptocorn in ~ λ GHIDRA_INSTALL_DIR=/opt/ghidra_12.0.4 pyrrha fs-cg RAX30_rootfs -j 12
Binaries mapping          ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 701/701 0:06:11
Symlinks mapping          ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 349/349 0:00:00
Library imports mapping   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 701/701 0:00:00
Call Graph mapping        ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 701/701 0:00:04
```

Decompiled file

pyrrha decomp

```
(pyrrha) cryptocorn in ~ λ IDADIR=/opt/ida-pro-9.3 pyrrha decomp RAX30_rootfs/usr/lib/libfwcheck.so
Function indexing      ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 230/230 0:00:01
Source indexing        ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 230/230 0:00:00
Call Graph indexing    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 230/230 0:00:00
```



Enum values

```
CURLOPT_SSL_VERIFYPEER = 64;
CURLOPT_SSL_VERIFYHOST = 81;
```

Try it

Pyrrha

- CLI: `pip install pyrrha-mapper`
- Backend: *IDA Pro 9.1+* or *Ghidra 12.0+*

https://github.com/quarkslab/pyrrha

NumbatUI

Several options:

- Docker
- Debian package
- From sources

https://github.com/quarkslab/NumbatUI

What Pyrrha will not do for you

- Results depend on the reverse-engineering backend — Ghidra and IDA do not recover the same call graph
- Structured firmware only: extractable filesystem, ELF binaries
- Best on C/C++ — symbols and mangled names are what make cross-binary matching possible

Quarkslab

REVERSE YOUR FUTURE

10 bd Haussmann | 75009 Paris
12 av. Henri Fréville | 35000 Rennes
A. Raúl Scalabrini Ortiz | 2444 | C1425DBS BUENOS AIRES

www.quarkslab.com

© Quarkslab 2026. All rights reserved.